Capstone Project: Hospital Readmission Risk Predictor

A phased MLOps build plan — Diabetes 130-US Hospitals dataset

Overview

Problem: Predict whether a patient will be readmitted within 30 days of discharge. Real-world relevance: hospitals get fined for high readmission rates.

Dataset: UCI Diabetes 130 Hospitals dataset (~100k patient records, free).

Why this over sepsis early-warning: Sepsis detection ends up being a straightforward classification task with no natural retrain/redeploy loop. Readmission risk supports a full ML system, which is the more valuable capstone to build.

Tech stack: Python-native (Airflow, MLflow, FastAPI — no real R equivalents for this architecture). JupyterLab works fine as the dev environment regardless of kernel.

Python level: Comfortable with scripts, pandas, sklearn. New to MLflow (small-medium lift), FastAPI (small lift), Airflow (real learning curve, mostly environment setup and DAG debugging).

Timeline: 3-6 weeks.

Dataset link: https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008
Represents ten years (1999–2008) of clinical care at 130 US hospitals, ~101,766 patient encounters, 50+ features, licensed CC BY 4.0, downloadable directly as CSV, no login required.
Citation: Clore, John, et al. "Diabetes 130-US Hospitals for Years 1999-2008." UCI Machine Learning Repository, 2014, https://doi.org/10.24432/C5230J.

Heads-up: The target column isn't a clean binary "readmitted or not" — it's typically three classes (<30, >30, NO). Collapse this into a binary target (<30 = 1, everything else = 0) during feature engineering, since the capstone is specifically about the 30-day window.

Model Requirement: One Supervised + Two Unsupervised

This is one dataset used for three different modeling tasks — not three separate datasets. No additional data sourcing is needed.

1. Supervised model (required)

The readmission classifier (logistic regression, random forest, or XGBoost) predicting the binary 30-day readmission label. This is the core model in the MLflow / FastAPI / Airflow pipeline.

2. Unsupervised model #1 — Clustering

K-means or hierarchical clustering to group patients into risk profiles/segments based on features like

age, number of medications, time in hospital, number of prior visits, and diagnosis codes. Useful for identifying natural patient groups and checking whether certain clusters correlate with higher readmission rates.

3. Unsupervised model #2 — Dimensionality Reduction

PCA (or t-SNE) to reduce 50+ features down to a handful of components capturing most of the variance. Useful for (a) visualizing whether patients naturally separate into groups in 2D, and (b) optionally using the reduced components as engineered features feeding into the supervised model — tying the unsupervised work directly into the main pipeline.

Where this fits: Both unsupervised models are exploratory/analysis work that lives in Phase 1, alongside feature engineering — before finalizing the supervised model. They don't touch the MLflow / FastAPI / Airflow architecture; that pipeline still only serves the one supervised classifier.

The Deal

Get Phase 1 fully working and solid first. That's the deliverable, full stop — not a rough draft to rush past. Once it's genuinely working end-to-end, then decide whether to push into Phase 2.

Phase 1 working = capstone done, no shame in stopping there.
Phase 1 working early = green light to go earn the harder version.

Phase 1 — Required Build (Weeks 1–3)

- Clean/engineer features on the Diabetes 130 dataset
- Train supervised model as a plain Python script — written with a callable main(), not notebook-only, so it's reusable later
- Run clustering and PCA as exploratory unsupervised analysis alongside feature engineering
- Log training runs to MLflow (params, metrics, model artifact)
- Manually promote best run to "Production" in the MLflow Model Registry
- Build a FastAPI endpoint that loads whatever model is tagged "Production" and serves live predictions

This alone is a complete, defensible capstone: train → track → serve, on a real problem, with class imbalance handling and real experiment tracking, plus supporting unsupervised analysis. If nothing else happens, this stands on its own.

Phase 2 — The Challenge (Weeks 4–6)

Once Phase 1 is solid and demoable, push further: wrap it into a system that keeps itself updated, rather than one you retrain by hand.

- Wrap the same Phase 1 training script in an Airflow DAG (retrain task)

- Add an evaluation task that compares the new model's metrics against the current Production model
- Add a conditional promote task — only flips the Production tag if the new model actually wins
- FastAPI needs zero changes — it already just reads whatever's tagged Production

Why this is the harder, more impressive version: it's the difference between "I trained a model" and "I built a system that retrains and redeploys itself only when it earns it." That's the MLOps signal that separates this from a standard classification project.

Tips for When You Get There

- Use docker-compose for Airflow setup — avoids most dependency hell
- Keep the DAG simple: 3-4 tasks (fetch/check data → retrain → evaluate → conditionally promote), not elaborate branching
- Budget real buffer time for Airflow debugging specifically — it's the most likely place to lose time
- Airflow should only orchestrate code you've already tested in Phase 1, never contain new logic itself